

SAFELearn: Secure Aggregation for private FEderated Learning

Hossein Fereidooni¹, Samuel Marchal², Markus Miettinen¹, Azalia Mirhoseini³, Helen Möllering⁴,
Thien Duc Nguyen¹, Phillip Rieger¹, Ahmad-Reza Sadeghi¹, Thomas Schneider⁴,
Hossein Yalame⁴, and Shaza Zeitouni¹

¹ System Security Lab, Technical University of Darmstadt, Germany

{hossein.fereidooni, markus.miettinen, duchien.nguyen, phillip.rieger, ahmad.sadeghi, shaza.zeitouni}@trust.tu-darmstadt.de

² Aalto University and F-Secure Corporation, Finland – samuel.marchal@aalto.fi

³ Google, USA – azalia@google.com

⁴ ENCRYPTO, Technical University of Darmstadt, Germany – {moellering, schneider, yalame}@encrypto.cs.tu-darmstadt.de

Abstract—Federated learning (FL) is an emerging distributed machine learning paradigm which addresses critical data privacy issues in machine learning by enabling clients, using an aggregation server (aggregator), to jointly train a global model without revealing their training data. Thereby, it improves not only privacy but is also efficient as it uses the computation power and data of potentially millions of clients for training in parallel.

However, FL is vulnerable to so-called *inference attacks* by malicious aggregators which can infer information about clients' data from their model updates. Secure aggregation restricts the central aggregator to only learn the summation or average of the updates of clients. Unfortunately, existing protocols for secure aggregation for FL suffer from high communication, computation, and many communication rounds.

In this work, we present SAFELearn, a generic design for efficient private FL systems that protects against inference attacks that have to analyze individual clients' model updates using secure aggregation. It is flexibly adaptable to the efficiency and security requirements of various FL applications and can be instantiated with MPC or FHE. In contrast to previous works, we only need 2 rounds of communication in each training iteration, do not use any expensive cryptographic primitives on clients, tolerate dropouts, and do not rely on a trusted third party. We implement and benchmark an instantiation of our generic design with secure two-party computation. Our implementation aggregates 500 models with more than 300K parameters in less than 0.5 seconds.

Index Terms—Federated Learning, Inference Attacks, Secure Computation, Data Privacy

I. INTRODUCTION

Federated Learning (FL) became a distributed machine learning (ML) paradigm since it was introduced by Google in 2017 [1]. It aims at improving privacy by enabling data owners to efficiently train a model on their joint training data with the help of a central aggregator and without sharing their potentially sensitive data with each other or with the aggregator. Possible applications include, e.g., next word prediction for mobile keyboards from Google [2], the analysis of medical data [3], communication between vehicles [4], and intrusion detection systems [5]. While FL leverages the power of the massive amount of data available at edge devices nowadays, it improves data privacy by enabling to keep data locally at the

clients [2]. This becomes particularly relevant not only because of legal obligations such as the GDPR [6] and HIPAA [7], but also in general when working with personal and sensitive data like in the health sector where ML gets increasing attention. In applications being deployed on end-users' devices, FL helps to increase the acceptance as the user's data never leaves its device such that more users might be willing to contribute to a training.

Despite these benefits, FL is vulnerable to adversarial attacks aiming at extracting information about the used training data. In these so-called *inference attacks*, the adversary can, for example, infer if a specific image was used in training an image classifier by inferring the *model updates* [8], [9]. This violates the principal design goal of FL, i.e., protecting data privacy. Several secure aggregation protocols have been proposed to address this problem by hindering the aggregator from analyzing clients' model updates [10]–[20]. However, existing approaches are inefficient, impractical, and/or rely on a trusted third party (TTP) [11]–[13]. In particular, they are computationally expensive [10], [16], [18], [21], increase the number of communication rounds [10], [19], and do not tolerate dropouts [11], [12]. Especially the increase in communication rounds is problematic, as FL is typically used in a mobile setting where mobile or edge devices are involved and the network tends to be unstable, slow, and with low bandwidth [1]. Mobile devices regularly go offline such that dropouts must be tolerated. Most importantly, these aggregation schemes hinder the aggregator from accessing the local updates, therefore, making it impossible to analyze these updates for malicious client behavior that sabotage the training [22].

Our Contributions and Outline. In this work, we introduce SAFELearn, an efficient secure aggregation system, prohibiting access to model updates to impede powerful inference attacks on FL. In particular, we provide the following contributions after giving the preliminaries in §II:

- We survey state-of-the-art secure aggregation protocols for FL and analyze their limitations (§III).
- We introduce a generic design called SAFELearn for secure aggregation for FL. It is adaptable to various secu-

urity and efficiency requirements and multiple aggregation mechanisms (§IV).

- We implement and benchmark an instantiation of SAFELearn using secure two-party computation on multiple FL applications and datasets (§V). Our system aggregates 500 models with more than 300K parameters in less than 0.5 seconds. Our implementation is available as open source at <https://github.com/TRUST-TUDa/SAFELearn>.

II. PRELIMINARIES

A. Federated Learning (FL)

Federated Learning (FL) [1], [23] is a concept for distributed machine learning that links K clients and an aggregator A who collaboratively build a global model G . In each training iteration t , A chooses a subset of the K clients and sends the current global model G_{t-1} to them. Instead of sharing gradients after each training iteration as in standard distributed machine learning, each of these clients $i \in K$ then trains G_{t-1} on multiple batches of its training data for multiple epochs before sending the resulting locally updated model W_i to the aggregator. Then, A aggregates the received updates W_i into the global model G_t . FL results in less global training iterations than in standard distributed machine learning and, hence, in less communication.

Several aggregation mechanisms have been proposed for FL: (1) *Federated-Averaging* (FedAvg) [1], (2) *Krum* [24], (3) *Adaptive Federated Averaging* [25], and (4) *Trimmed mean or median* [26]. In this work, we focus on *FedAvg*, which is the original FL aggregation mechanism, because it is commonly applied in FL and related work on secure aggregation [10], [15]–[18]. In FedAvg, the global model is updated by summing the weighted (by the number of training samples used to train it) models $G_t = \sum_{i=1}^{|K|} \frac{s_i \times W_i}{s}$, where K is the set of clients, $s_i = \|D_i\|$ for training data D_i of a client $i \in K$, $s = \sum_{i=1}^{|K|} s_i$, and W_i is client i 's update [1].

B. Inference Attacks on FL

In an *inference attack*, an adversary aims at learning information about the data used for training a ML model. *Membership inference* attacks determine whether certain samples were used for training [27], *property inference* attacks infer properties of training samples independent of the original learning task [9], *distribution estimation* attacks estimate the proportions of training labels in the data [28], and *reconstruction* attacks reconstruct training samples [28]. Another distinction can be made between black box attacks, that are restricted to interpret the model predictions [28], and white box attacks, that use model parameters of either the trained model or from the clients' updates during the training [27], [28].

FL protects the privacy of the clients' data against inference attacks run by third parties, as they can only access the global model and cannot relate the information inferred from a global model to a specific client. Additionally, attacks on the global models tend to be weak and fail to achieve good attack performance [27]. However, the aggregator in FL has access to the local updates of each client making FL vulnerable to

strong inference attacks by a corrupted aggregator. Thus, in this work, we aim at hindering the aggregator from accessing clients' update to prohibit powerful inference attacks that are leveraging individual local updates of clients while enabling efficient FL.

C. Secure Multi-Party Computation (MPC)

Secure Multi-Party Computation (MPC) enables the secure evaluation of a public function on private data provided by N mutually distrusting parties [29].

Secure two-party computation (STPC) [30]–[33], a special case of MPC with two parties ($N = 2$), allows two parties to securely evaluate a function on their private inputs.

Thereby, the parties have only access to so-called secret-shares of the inputs that are completely random and therefore do not leak any information. The real value can only be obtained if both shares are combined. STPC can be used in an outsourcing scenario [31], where an arbitrary number of weak but even malicious clients can secret-share their private inputs among two non-colluding but well-connected and powerful servers who then run the STPC protocol.

D. Homomorphic Encryption (HE)

Homomorphic Encryption (HE) enables computations on encrypted data. It allows to perform operations on a ciphertext, the decryption of which corresponds to algebraic operations on the plaintext. HE schemes can be classified into partially (PHE), somewhat (SHE), or fully homomorphic encryption (FHE). PHE [34] supports either multiplication or addition under encryption, SHE supports a bounded number of both, and FHE [35] supports both without limitations.

E. Differential Privacy

Although Differential Privacy (DP) [36] is not the focus of our work, we shortly summarize it here for the sake of completeness as it is used in some related works (cf. §III).

Informally, DP [36] randomizes the result of an evaluation (e.g., by adding noise) to reduce information leakage. More formally, a randomized algorithm $\mathcal{M}$ with domain $\mathcal{D}$ satisfies (ϵ, δ) -DP if for all adjacent datasets $d, d^* \in \mathcal{D}$ and for all $S \subseteq \text{Range}(\mathcal{M})$ it holds, that $\Pr[\mathcal{M}(d) \in S] \leq e^\epsilon \Pr[\mathcal{M}(d^*) \in S] + \delta$.

III. RELATED WORK

Tab. I shows a comparison between SAFELearn and previous works with respect to their usability and their privacy guarantees, more details in [38, Tab. 1].

A. Secure Aggregation for FL with Secret Sharing

Bonawitz et al. [10] introduced secure aggregation for FL. Their protocol can tolerate client dropouts. They use blinding with random values, Shamir's Secret Sharing (SSS) [39], and symmetric encryption to prohibit access to local models. However, their aggregation requires at least 4 communication rounds between each client and the aggregator in each iteration. This causes a significant overhead for clients typically connected via WAN and with limited resources.

TABLE I

COMPARISON OF PRIVACY-PRESERVING FL FRAMEWORKS. OUR PRIVACY ANALYSIS INCLUDES THE INVOLVEMENT OF A TRUSTED THIRD PARTY AND IF THE SCHEMES ARE EXTENDABLE TO ACTIVE SECURITY. OUR USABILITY ANALYSIS INCLUDES THE COMMUNICATION ROUND-EFFICIENCY, ROBUSTNESS TO DYNAMIC CLIENT DROPOUT, THE USAGE OF EXPENSIVE CRYPTOGRAPHIC OPERATIONS, AND THE AVAILABILITY OF OPEN-SOURCE CODE.

Framework	Privacy		Usability			
	No Trusted Party	Extend-to-Active	Round-efficient	Dropout	No Expensive Operations	Open-Source
Truex et al. [11]	✗	✗	✗	✗	✗	✗
HybridAlpha [12]	✗	✗	✗	✓	✗	✗
Bonawitz et al. [10]	✓	✓	✗	✓	✗	✗
BatchCrypt [21]	✓	✗	✓	✓	✗	✓
VeriFL [20]	✓	✗	✗	✓	✗	✗
Choi et al. [17]	✓	✗	✗	✓	✗	✗
FastSecAgg [18]	✓	✗	✗	✓	✗	✗
SAFER [15]	✓	✓	✓	✗	✓	✗
POSEIDON [37]	✓	✓	✓	✓	✗	✗
Bell et al. [16]	✓	✓	✗	✓	✗	✗
Turbo-Aggregate [19]	✓	✓	✗	✓	✗	✗
SAFElearn (This work)	✓	✓	✓	✓	✓	✓

VerifyNet [13], and VeriFL [20] use the protocol of Bonawitz et al. [10]. VerifyNet and VeriFL add verifiability on top of [10] to guarantee the correctness of the aggregation, but these protocols rely on a trusted party to generate public/private key pairs for all clients.

Recently, the authors of [16] and [17] introduced secure aggregation protocols with polylogarithmic communication and computation complexity which reduce the overhead compared to [10]. Their key idea is to replace the star topology of the communication network in [10] by random subgroups of clients and to use secret sharing only for a subset of clients instead of for all client pairs. Both approaches require 3 rounds of interaction between the server and clients.

FastSecAgg [18] provides a secure aggregation based on the Fast Fourier Transform multi-secret sharing. It is robust against adaptive adversaries where the clients can adaptively be corrupted during the execution of the protocol. FastSecAgg is a 3 round interactive protocol for private FL.

Turbo-Aggregate [19] reduces the communication and computation overhead of secure aggregation over [10] (cf. Tab. II) and uses a circular communication topology. The main bottleneck of Turbo-Aggregated is its $\mathcal{O}(n/\log n)$ round complexity, where n is the number of updates/clients (cf. §IV).

SAFER [15] reduces communication costs in FL by compressing updates and combines it with a secure aggregation protocol based on arithmetic sharing. However, SAFER considers only training with less than 10 clients and no dropouts. Moreover, SAFER was only benchmarked on independent and identically distributed (IID) data such that it is unclear if it works with the typically non-IID data used in FL.

B. Secure Aggregation for FL with Encryption

Truex et al. [11] combine additively homomorphic encryption (HE) with DP but cannot tolerate client dropouts. Using HE results in a significant runtime overhead and their system also requires 3 rounds of communication. These aspects make it impractical for real-world FL.

EaSTffly [14] uses either additively HE with packing or Shamir’s secret sharing (SSS) [39] in combination with

quantization. The clients share their gradients after each training iteration instead of using FL’s FedAvg mechanism [10] which significantly increases the number of training iterations. FedAvg requires division which is not possible with additively HE/SSS. Furthermore, in EaSTffly’s HE protocol all clients have to hold the same secret key such that if a client colludes with the aggregator all updates can be decrypted.

BatchCrypt [21] reduces the encryption and communication overhead of HE-based aggregation with a batch encryption technique and requires only a single round of communication. Again, using expensive HE (like [11], [14]) makes it unusable for real-world training with FL.

HybridAlpha [12] uses functional encryption and DP. With functional encryption, public keys for all clients are derived from a private/public master key pair. It improves [11]’s runtime by a factor of $2\times$ and tolerates dropouts. However, HybridAlpha relies on a trusted party that holds the master keys and controls if the aggregator manipulates the aggregation weights.

POSEIDON [37] encrypts the complete FL process including the local training executed by the clients and, thus, adds significant computational overhead on each client’s device. The authors suggest to reduce the clients’ communication by combining them in a tree-like network instead of the classical star topology where each client directly communicates with the central aggregator. Additionally, a distributed bootstrapping efficiently refreshes ciphertexts and an alternating packing approach enhances the efficiency of neural network training under encryption. However, POSEIDON only supports clients’ dropouts when the decentralized bootstrapping is not used.

Generally, all existing protocols for secure aggregation hinder the aggregators from deploying defenses against so-called *backdooring* or *poisoning attacks* [22], [40] that aim at injecting a “backdoor” into the ML model, i.e., the model is manipulated such that it misclassifies a small set of attacker-chosen inputs as attacker-chosen outputs. Bagdasaryan et al. [22] show, inter alia, how a single client can manipulate FL by injecting a backdoor that causes green cars to be misclassified as birds. Such targeted image misclassification can, for example, be dangerous for face recognition systems deployed at airports.

FLGUARD [41] and BaFFLe [42] combine secure aggregation with defenses against backdoor injections.

IV. PRIVATE FEDERATED LEARNING

SAFELearn has three major *design goals* to overcome the limitations of previous work (cf. §III): (G1) prohibiting access to individual model updates to counter powerful inference attacks, (G2) efficiency, and (G3) tolerance to outliers.

A. Adversary Model — Goals and Capabilities

The adversary is a semi-honest¹ aggregator such that we assume it follows the protocol honestly, but attempts to infer sensitive information about clients' data D_i from their model updates W_i [43], [44].

In the standard FL setting, the aggregator has access to all local model updates W_i , such that it can perform model inference attacks on each local model to extract information about the corresponding participant's data D_i used for training. Some existing attacks, e.g., [45], consider that the adversary (either a semi-honest aggregator or client) aims at inferring information about training data from the global model G_t . However, these attacks obtain negligible aggregated information about data (and cannot link it to individual clients) like the number of classes [45]. For example, Nasr et al. [27] show that the success of membership inference attacks on only the global model significantly degrades with an increasing number of clients. Therefore, our goal is to hide local models from the aggregator to impede powerful inference attacks while still enabling efficient and accurate FL.

B. SAFELearn

The simple and generic design of SAFELearn that realizes the adapted FedAvg with equal weights (cf. §II) in a private manner is depicted in Alg. 1. It takes the set of clients K , the initial global model G_0 , and the number of training iterations T as input. Then, in each iteration $t \in [1, T]$, a random subset of clients $K_t \subseteq K$ is chosen following the original design of FL [1]. Each client $i \in K_t$ receives the encrypted/secret shared global model $[G_{t-1}]$ from the aggregator(s) which it decrypts to train a new updated local model W_i . The client sends encrypted/secret shared update $[W_i]$ to the aggregator(s) for the aggregation of a new global model $[G_t]$ in Line 6.

Depending on the efficiency and security requirements of the concrete application, SAFELearn can be realized with fully homomorphic encryption (FHE), multi-party computation (MPC), or secure two-party computation (STPC). These secure computation techniques ensure that the aggregator cannot access clients' model updates and intermediate global models to effectively thwart powerful inference attacks (G1, cf. §IV).

1) *FHE*: If FHE with a *single* semi-honest server (acting as the aggregator) is used, $[G_t]$ and $[W_i]$ are the encryption of the models' parameters. The clients use a multi-party encryption scheme [46] for encrypting their updates. The secret key is securely split among the clients such that each of them can decrypt and access the global updates in plaintext for

¹Can be extended to active security, cf. §V.

Algorithm 1 SAFELearn

```

1: Input:  $K, G_0, T \triangleright K$ : set of clients,  $G_0$ : initial global model,
    $T$ : # iterations
2: Output:  $G_T \triangleright G_T$  is the global model after  $T$  iterations
3: for each training iteration  $t \in [1, T]$  do
4:   for each client  $i \in K_t \subseteq K$  do  $\triangleright K_t \subseteq K$  is randomly
     chosen in every iteration.
5:      $[W_i] \leftarrow \text{CLIENTUPDATE}([G_{t-1}])$ 
6:   end for
7:    $[G_t] \leftarrow \sum_{i=1}^{|K_t|} [W_i] / |K_t|$ 
8: end for

```

local training. The clients then re-encrypt the resulting local model updates and return it to the server who aggregates the encrypted data to the new global model $[G_t]$ using the additive homomorphic properties of the encryption scheme.

2) *MPC/STPC*: If MPC/STPC is used, $[G_t]$ and $[W_i]$ are a set of secret shares of the models created with the Arithmetic Sharing technique [47] (cf. §V-A) and held by the $N \geq 2$ non-colluding servers. They jointly run the secure aggregation on these shares (i.e., the N servers together compose the aggregator). After the secure aggregation, the servers send the secret shares of the new global models to the clients who combine them to receive the plaintext global model for the next local training iteration. Afterwards, they again secretly share their updates and send one share to each server.

3) *Secure Aggregation in SAFELearn*: While we used FedAvg as an example in Line 6 of Alg. 1 given its popularity in the FL literature (cf. §II-A), different kinds of aggregation can also be realized with SAFELearn. Concretely, MPC and STPC support arbitrary computations expressed as Boolean circuit. Hence, also different aggregation mechanisms can be realized in a straightforward fashion. For example, the aggregation mechanism of Krum [24] consists of Argmin, multiplication, and addition operations. These can be realized by combination of different MPC/STPC protocols: Boolean sharing for the secure evaluation of Argmin and Arithmetic sharing for the secure evaluation of multiplication and addition operations. Similarly also other aggregation mechanisms (e.g., [25], [26]) can be realized by combining different MPC/STPC techniques.

C. Privacy & Usability of SAFELearn

SAFELearn needs only 2 rounds of communication per iteration (Line 5). It allows an arbitrary number of clients to drop out and rejoin as the aggregator(s) can simply adjust the division factor $|K_t|$ by the number of clients that respond to $\text{CLIENTUPDATE}(\cdot)$. Thus, SAFELearn offers efficiency (G2, §IV) with respect to the number of communication rounds² and tolerance to outliers (G3, §IV).

The aggregating server(s) do only learn the number of clients in each training iteration and, hence, SAFELearn effectively prevents the individual aggregation server(s) from running inferences attacks on the clients' local model updates. Moreover,

²Its efficiency w.r.t. communication and computation heavily depends on its concrete instantiation and cannot be generally assessed. We benchmark it for an instantiation with STPC in §V to show its practicality.

the aggregator(s) only hold secret shared or encrypted global models such that they also cannot run inference attacks on the global model. Even when the adversary controls a client, who still has access to the plaintext global model parameters, the aggregation of the models averages the parameters of all contributing clients and, thus, makes inference attacks harder and less effective [27]. Additionally, information extracted from a global model cannot be linked to a single client. Therefore, SAFElearn supports the anonymisation of the individual contributions.

To summarize, SAFElearn is a generic system for secure aggregation in FL and supports a wide range of applications by choosing the number of servers based on the specific security and efficiency requirements. It addresses all design goals (G1-G3) from the beginning of §IV. FHE/MPC/STPC support many operations such as addition, multiplication, and comparison. Those atomic operations also enable to privately realize aggregation functions beyond FedAvg, e.g., Krum [24].

V. EXPERIMENTAL EVALUATION OF SAFELEARN WITH STPC

Our framework SAFElearn is generic and can be instantiated with one (FHE) or multiple non-colluding servers (MPC/STPC). We implement one instantiation of SAFElearn using STPC as it is often a good trade-off between security and efficiency. We securely outsource the computation of the SAFElearn algorithm to two servers that are (1) non-colluding and (2) semi-honest. These properties and assumptions are described and justified next.

1) *Non-colluding semi-honest servers*: We assume that the two servers are *non-colluding*. In our envisioned FL applications mentioned in §I, the two non-colluding servers could, e.g., be run by two competing antivirus software companies for network intrusion detection or by two competing smartphone manufacturers for word prediction. These parties are assumed to not collude to protect their business secrets and customers' data. Moreover, the two servers are assumed to be *semi-honest*, meaning that they honestly follow the protocol, but seek to learn as much information as possible. Service providers, like antivirus companies or smartphone manufacturers, have an inherent motivation to follow the protocol because they want to offer a privacy-preserving service to their customers and if cheating would be detected, this would seriously damage their reputation, which is the foundation of their business models.

2) *Two servers*: We choose $N = 2$ servers, i.e., STPC, as a reasonable trade-off between security and efficiency: With only one server, we would not need the non-collusion assumption, but this requires very expensive cryptographic primitives like fully homomorphic encryption [35] or several rounds of interaction with the clients. As discussed in §I, it is beneficial to minimize the number of communication rounds because of the mobile setting with unstable and slow connections between clients and the aggregator in which FL is typically used.

Using protocols such as [48] with three or even more non-colluding servers of which at most one can be corrupted allows

to construct even more efficient protocols than with two servers, but this has a larger attack surface because an attacker can attack any of the $N \geq 3$ servers and also more non-colluding parties have to be found to run these servers. However, our implementation can be easily extended to 3 semi-honest servers by using the ABY³ [48] framework, N semi-honest servers by using the MOTION [29] framework, or p malicious servers by using MP-SPDZ [49].

A. Benchmarks

For our instantiation, we use a combination of two STPC techniques, which are implemented with state-of-the-art optimizations in the ABY framework [31]: Boolean sharing using Yao's garbled circuits [30] for secure evaluation of Boolean division circuits in a constant number of rounds, as well as Arithmetic sharing for secure evaluation of additions using the GMW protocol of Goldreich-Micali-Wigderson [47]. We use the PyTorch framework [50] for neural network training. All STPC results are averaged over 10 experiments and run on two separate servers with Intel Core i9-7960X CPUs with 2.8 GHz and 128 GB RAM connected over a 10 Gbit/s LAN with 0.2 ms RTT.

B. Applications

We test SAFElearn on three datasets for typical FL applications:

1) *Natural Language Processing (NLP)*: We use a recurrent neural network with 20M parameters from two long short-term memory (LSTM) and one linear output layer [22]. In each iteration t , $K_t = 100$ clients are randomly chosen to train the model. We use the Reddit dataset from November 2017 [51] with 20.6M records. Each Reddit user with at least 150 posts and less than 500 posts is considered as a FL client. We generated a dictionary based on the most frequent 50 000 words.

2) *Image Classification (IC)*: Following [22], we used the CIFAR-10 dataset [52] with 50 000 images and a lightweight version of ResNet-18 with 2.7M parameters from 4 convolutional layers and a fully connected output layer. We split the dataset among 100 clients as done in [22]. In each training iteration, the clients locally update the model with a learning rate of 0.1.

3) *Network Intrusion Detection System (NIDS)*: We use the IoT NIDS DIoT [5] with three datasets by [5], [53] and one self-collected dataset from homes and offices located in Germany and Australia. Following [5], we extracted device-type-specific datasets capturing the communication behavior of a smart weather station. We simulate the FL setup by splitting the data among 106 clients, each having three hours of traffic measurements and select 100 clients from them at random in each training iteration. The learning rate is 0.1.

C. Impact on the the accuracy of the resulting model

To measure SAFElearn's impact on the model's accuracy, we run experiments on all three datasets presented in §V-B. Tab. III shows our results as well as the experimental setup, including the number of local training epochs, the number of

TABLE II
COMPUTATION, COMMUNICATION, AND COMMUNICATION ROUNDS (BETWEEN SERVER AND CLIENTS) PER TRAINING ITERATION OF SAFELEARN AND RELATED WORKS BASED ON SECRET SHARING. HERE n IS THE TOTAL NUMBER OF LOCAL MODELS (I.E., NUMBER OF CLIENTS) AND m IS THE LENGTH OF MODEL UPDATES. BEST MARKED IN BOLD.

Approach	Computation (Server)	Communication (Server)	Computation (Client)	Communication (Client)	Rounds
Turbo-Aggregate [19]	$\mathcal{O}(m \log n \log^2 \log n)$	$\mathcal{O}(mn \log n)$	$\mathcal{O}(m \log n \log^2 \log n)$	$\mathcal{O}(m \log n)$	$n / \log n$
Bonawitz et al. [10]	$\mathcal{O}(mn^2)$	$\mathcal{O}(mn + n^2)$	$\mathcal{O}(mn + n^2)$	$\mathcal{O}(m + n)$	4
Bell et al. [16]	$\mathcal{O}(mn \log n + n \log^2 n)$	$\mathcal{O}(mn + n \log n)$	$\mathcal{O}(m \log n + \log^2 n)$	$\mathcal{O}(m + \log n)$	3
FastSecAgg [18]	$\mathcal{O}(m \log n)$	$\mathcal{O}(mn + n^2)$	$\mathcal{O}(m \log n)$	$\mathcal{O}(m + n)$	3
Choi et al. [17]	$\mathcal{O}(mn \log n)$	$\mathcal{O}(n\sqrt{n} \log n + mn)$	$\mathcal{O}(n \log n + m\sqrt{n} \log n)$	$\mathcal{O}(\sqrt{n} \log n + m)$	3
SAFElearn (This work)	$\mathcal{O}(mn)$	$\mathcal{O}(mn)$	$\mathcal{O}(m)$	$\mathcal{O}(m)$	2

previously trained rounds and the total number of clients for that dataset, from which a subset of 100 clients is randomly chosen to perform the training in each training iteration. The results show that SAFElearn has the same accuracy as plaintext FedAvg.

TABLE III
EXPERIMENTAL SETUP AND ACCURACY OF FEDAVG AND SAFELEARN FOR THE FL APPLICATIONS NATURAL LANGUAGE PROCESSING (NLP), IMAGE CLASSIFICATION (IC), AND NETWORK INTRUSION DETECTION SYSTEM (NIDS).

	NLP	IC	NIDS
Local Epochs	250	2	10
Pretrained Rounds	5 000	10 000	10
Available Clients	80 000	100	106
FedAvg	22.5%	91.7%	100.0%
SAFElearn (This work)	22.5%	91.7%	100.0%

D. Efficiency of SAFElearn

The results of our efficiency evaluation of SAFElearn with STPC between the two servers for the three datasets with different numbers of clients per training iteration, ranging from 10 to 500, are shown in Figs. 1 and 2. The runtime scales linearly with the number of clients and the communication is about constant. For NIDS, aggregating 500 models takes 0.5 seconds and the communication between the two servers is 8 MB. Even for the very large NLP model with more than 20M parameters, the aggregation takes less than 80 seconds and 316 MB between the two servers.

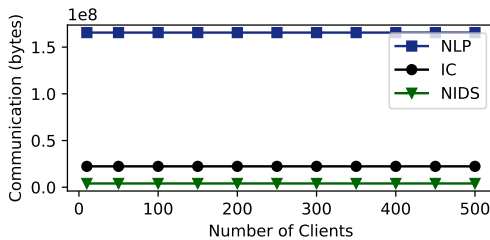


Fig. 1. Communication Costs per Server in SAFElearn

E. Analytical Complexity

Tab. II shows the substantially improved complexities of SAFElearn over the five previous works on secure aggregation for FL that consider dropouts and are not based on computationally expensive HE [10], [16]–[19].

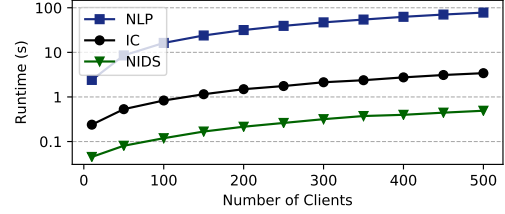


Fig. 2. Total Execution Time of SAFElearn

VI. CONCLUSION

In this paper, we introduce SAFElearn, a generic private federated learning design that enables to efficiently thwart strong inference attacks that need access to clients' individual model updates. Moreover, SAFElearn tolerates dropouts and does not require expensive cryptographic operations, making it more efficient than previous works with respect to communication and computation. Furthermore, it does not rely on a trusted third party. Our evaluation shows that aggregating 500 models with more than 300K parameters takes less than 0.5 seconds on commodity hardware.

Future work can realize more instantiations of SAFElearn. Also the combination of privacy and security in FL which was considered to be contradicting by Bagdasaryan et al. [22] can be investigated. SAFElearn's design could also enable to integrate a defense against manipulations of malicious clients.

Acknowledgements. This project received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program (grant agreement No. 850990 PSOTI), was co-funded by the DFG — SFB 1119 CROSSING/236615297 and GRK 2050 Privacy & Trust/251805230, and by the BMBF and HMWK within ATHENE. It was partially funded by the European Commission through the SHERPA Horizon 2020 project under grant agreement No. 786641. It was partially funded by the Private AI Collaborative Research Institute (PrivateAI) established by Intel, Avast, and Borsetta.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *International Conference on Artificial Intelligence and Statistics*, 2017.

- [2] B. McMahan and D. Ramage, "Federated Learning: Collaborative Machine Learning without Centralized Training Data," in *Google Research Blog*. Google AI, 2017, <https://ai.googleblog.com/2017/04/federated-learning-collaborative.html>.
- [3] T. S. Brisimi, R. Chen, T. Mela, A. Olshevsky, I. C. Paschalidis, and W. Shi, "Federated Learning of Predictive Models from Federated Electronic Health Records," *International Journal of Medical Informatics*, 2018.
- [4] S. Samarakoon, M. Bennis, W. Saad, and M. Debbah, "Federated Learning for Ultra-Reliable Low-Latency V2V Communications," in *GLOBECOM*, 2018.
- [5] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DfIoT: A Federated Self-learning Anomaly Detection System for IoT," in *ICDCS*, 2019.
- [6] "General Data Protection Regulation," 2018, <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [7] "Health Insurance Portability and Accountability Act," 1996, <https://www.govinfo.gov/content/pkg/PLAW-104publ191/pdf/PLAW-104publ191.pdf>.
- [8] N. Carlini, C. Liu, Ú. Erlingsson, J. Kos, and D. Song, "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks," in *USENIX Security*, 2019.
- [9] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting Unintended Feature Leakage in Collaborative Learning," in *S&P*, 2019.
- [10] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical Secure Aggregation for Privacy-preserving Machine Learning," in *CCS*, 2017.
- [11] S. Truex, N. Baracaldo, A. Anwar, T. Steinke, H. Ludwig, and Y. Zhou, "A Hybrid Approach to Privacy-preserving Federated Learning," in *AISec*, 2019.
- [12] R. Xu, N. Baracaldo, Y. Zhou, A. Anwar, and H. Ludwig, "Hybridalpha: An Efficient Approach for Privacy-preserving Federated Learning," in *AISec*, 2019.
- [13] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and Verifiable Federated Learning," in *Transactions on Information Forensics and Security*, 2020.
- [14] Y. Dong, X. Chen, L. Shen, and D. Wang, "EaSTFLy: Efficient and Secure Ternary Federated Learning," in *Computers & Security*, 2020.
- [15] C. Beguier and E. Tramel, "SAFER: Sparse Secure Aggregation for Federated Learning," 2020, <https://arxiv.org/abs/2007.14861>.
- [16] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure Single-Server Aggregation with (Poly)logarithmic Overhead," in *CCS*, 2020.
- [17] B. Choi, J.-y. Sohn, D.-J. Han, and J. Moon, "Communication-Computation Efficient Secure Aggregation for Federated Learning," 2020, <https://arxiv.org/abs/2012.05433>.
- [18] S. Kadhe, N. Rajaraman, O. O. Koyluoglu, and K. Ramchandran, "Fast-SecAgg: Scalable Secure Aggregation for Privacy-Preserving Federated Learning," *ICML Workshop on Federated Learning for User Privacy and Data Confidentiality*, 2020.
- [19] J. So, B. Güler, and A. S. Avestimehr, "Turbo-Aggregate: Breaking the Quadratic Aggregation Barrier in Secure Federated Learning," *Journal on Selected Areas in Information Theory*, 2021.
- [20] X. Guo, Z. Liu, J. Li, J. Gao, B. Hou, C. Dong, and T. Baker, "VeriFL: Communication-Efficient and Fast Verifiable Aggregation for Federated Learning," *TIFS*, 2020.
- [21] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "BatchCrypt: Efficient Homomorphic Encryption for Cross-Silo Federated Learning," in *USENIX ATC*, 2020.
- [22] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to Backdoor Federated Learning," in *AISTATS*, 2020.
- [23] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konecny, S. Mazzocchi, B. McMahan, T. V. Overveldt, D. Petrou, D. Ramage, and J. Roselander, "Towards Federated Learning at Scale: System Design," in *SysML*, 2019.
- [24] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent," in *NIPS*, 2017.
- [25] L. Muñoz-González, K. T. Co, and E. C. Lupu, "Byzantine-Robust Federated Machine Learning through Adaptive Model Averaging," 2019, <https://arxiv.org/abs/1909.05125>.
- [26] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates," in *ICML*, 2018.
- [27] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive Privacy Analysis of Deep Learning: Passive and Active White-box Inference Attacks against Centralized and Federated Learning," in *S&P*, 2019.
- [28] A. Salem, A. Bhattacharya, M. Backes, M. Fritz, and Y. Zhang, "Updates-leak: Data Set Inference and Reconstruction Attacks in Online Learning," in *USENIX Security*, 2020.
- [29] L. Braun, D. Demmler, T. Schneider, and O. Tkachenko, "MOTION-A Framework for Mixed-Protocol Multi-Party Computation," 2020.
- [30] A. C.-C. Yao, "How to Generate and Exchange Secrets," in *FOCS*, 1986.
- [31] D. Demmler, T. Schneider, and M. Zohner, "ABY - A Framework for Efficient Mixed-Protocol Secure Two-Party Computation," in *NDSS*, 2015.
- [32] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "ABY2. 0: Improved Mixed-protocol Secure Two-party Computation," in *USENIX Security*, 2020.
- [33] H. Yalame, H. Farzam, and S. Bayat-Sarmadi, "Secure Two-Party Computation Using an Efficient Garbled Circuit by Reducing Data Transfer," in *Applications and Techniques in Information Security*, 2017.
- [34] P. Paillier, "Public-key Cryptosystems Based on Composite Degree Residuosity Classes," in *EUROCRYPT*, 1999.
- [35] C. Gentry, "A Fully Homomorphic Encryption Scheme," Ph.D. dissertation, Stanford University, 2009.
- [36] C. Dwork and A. Roth, "The Algorithmic Foundations of Differential Privacy," in *Foundations and Trends in Theoretical Computer Science*, 2014.
- [37] S. Sav, A. Pyrgelis, J. R. Troncoso-Pastoriza, D. Froelicher, J.-P. Bossuat, J. S. Sousa, and J.-P. Hubaux, "POSEIDON: Privacy-Preserving Federated Neural Network Learning," in *NDSS*, 2021.
- [38] H. Fereidooni, S. Marchal, M. Miettinen, A. Mirhoseini, H. Möllering, T. D. Nguyen, P. Rieger, A. R. Sadeghi, T. Schneider, H. Yalame, and S. Zeitouni, "SAFElearn: Secure Aggregation for private Federated Learning (Full Version)," 2021, <https://ia.cr/2021/XXX>.
- [39] A. Shamir, "How to Share a Secret," in *Communications of the ACM*, 1979.
- [40] T. D. Nguyen, P. Rieger, M. Miettinen, and A.-R. Sadeghi, "Poisoning Attacks on Federated Learning-Based IoT Intrusion Detection System," in *Workshop on Decentralized IoT Systems and Security @ NDSS*, 2020.
- [41] T. D. Nguyen, P. Rieger, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen, A. Mirhoseini, A.-R. Sadeghi, T. Schneider, and S. Zeitouni, "FLGUARD: Secure and Private Federated Learning," 2021, <https://ia.cr/2021/025>.
- [42] S. Andreina, G. A. Marson, H. Möllering, and G. Karame, "BaFFLe: Backdoor Detection via Feedback-based Federated Learning," in *ICDCS*, 2021.
- [43] A. Pyrgelis, C. Troncoso, and E. De Cristofaro, "Knock Knock, Who's There? Membership Inference on Aggregate Location Data," in *NDSS*, 2018.
- [44] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership Inference Attacks Against Machine Learning Models," in *S&P*, 2017.
- [45] L. Wang, S. Xu, X. Wang, and Q. Zhu, "Eavesdrop the Composition Proportion of Training Labels in Federated Learning," 2019, <https://arxiv.org/abs/1910.06044>.
- [46] C. Mouchet, J. R. Troncoso-Pastoriza, and J.-P. Hubaux, "Multiparty Homomorphic Encryption from Ring-Learning-With-Errors," 2020, <https://ia.cr/2020/304>.
- [47] O. Goldreich, S. Micali, and A. Wigderson, "How to Play ANY Mental Game," in *STOC*, 1987.
- [48] P. Mohassel and P. Rindal, "ABY³: A Mixed Protocol Framework for Machine Learning," in *CCS*, 2018.
- [49] M. Keller, "MP-SPDZ: A Versatile Framework for Multi-Party Computation," in *CCS*, 2020.
- [50] "Pytorch," 2019, <https://pytorch.org>.
- [51] "Reddit dataset," 2017, https://bigquery.cloud.google.com/dataset/fh-bigquery:reddit_comments.
- [52] A. Krizhevsky and G. Hinton, "Learning Multiple Layers of Features from Tiny Images," Tech. Rep., 2009, <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- [53] A. Sivanathan, H. H. Gharakheili, F. Loi, A. Radford, C. Wijenayake, A. Vishwanath, and V. Sivaraman, "Classifying IoT Devices in Smart Environments Using Network Traffic Characteristics," in *Transactions on Mobile Computing*, 2018.